



PL-300

Search

CTRL

K



DAX-Berechnungen erstellen

Enders Training

CONTENTS

Berechnete Tabellen in der Praxis

Eine Tabelle duplizieren

Eine Datumstabelle erstellen

Als Datumstabelle markieren

Berechnete Spalten in der Praxis

Beispiele: Datumstabelle erweitern

Zeilenkontext verstehen

Implizite Measures

Aggregationsfunktionen

Einschränkungen impliziter Measures

Explizite Measures erstellen

Einfache Measures

Verbundmeasures

Quickmeasures

Berechnete Spalten vs. Measures — der wichtigste Unterschied

Iteratorfunktionen

Einfacher Vergleich: SUM vs. SUMX

Komplexe Zusammenfassung über mehrere Spalten

Granularität steuern mit AVERAGEX

Modul 3 · Skript 11

DAX-Berechnungen erstellen

Berechnete Tabellen & Spalten in der Praxis, implizite und explizite Measures, Iteratorfunktionen

Die Skripte 09 und 10 haben die Grundlagen von DAX gelegt — Berechnungstypen, Syntax, Datentypen, Funktionen und Operatoren. In diesem Skript geht es um die **praktische**

Umsetzung: Wie erstellt man berechnete Tabellen und Spalten konkret, wie funktionieren implizite und explizite Measures im Detail, und was leisten Iteratorfunktionen?

Berechnete Tabellen in der Praxis

Eine Tabelle duplizieren

Eine typische Herausforderung beim Modellbau ist der Umgang mit mehreren Beziehungen zwischen zwei Tabellen — z. B. wenn eine Faktentabelle mehrere Datumsspalten enthält. Da zwischen zwei Tabellen nur **eine aktive Beziehung** existieren kann, lässt sich das Problem durch das Duplizieren der Dimensionstabelle lösen.

Beispiel Die Tabelle *Sales* enthält drei Datumsspalten: Bestelldatum, Versanddatum und Fälligkeitsdatum — alle verknüpft mit der Tabelle *Date*. Nur eine Beziehung kann aktiv sein. Durch Duplizieren der Datumentabelle lassen sich separate Tabellen *Ship Date* und *Due Date* anlegen, die jeweils eine eigene aktive Beziehung zur Faktentabelle haben.

```
Ship Date = 'Date'
```

Diese Formel erzeugt eine neue Tabelle **Ship Date** mit denselben Spalten und Zeilen wie **Date**. Wird die Quelltable **Date** aktualisiert, wird **Ship Date** automatisch neu berechnet — die Tabellen bleiben synchron.

Screenshot Dateiname: dax-sales-date-relationships.png — Modelldiagramm: Tabelle Sales mit drei Beziehungen zur Tabelle Date — eine aktiv (durchgezogene Linie), zwei inaktiv (gestrichelte Linien).

Nach dem Duplizieren: Alle benutzerdefinierten Konfigurationen müssen auf die neue Tabelle übertragen werden — z. B. Spalten umbenennen, Sortierreihenfolge festlegen, nicht benötigte Spalten ausblenden.

Wichtig Berechnete Tabellen erhöhen die Modellgröße und können Datenaktualisierungszeiten verlängern, insbesondere wenn sie von anderen Tabellen abhängen. Sparsam einsetzen.

Eine Datumstabelle erstellen

Datumstabellen sind für DAX-Zeitintelligenzfunktionen zwingend erforderlich. Wenn die Quelldaten keine Datumstabelle enthalten, lässt sie sich mit DAX als berechnete Tabelle erstellen.

CALENDARAUTO() scannt automatisch alle Datums- und Datum/Uhrzeit-Spalten im Modell, ermittelt das früheste und späteste Datum und erzeugt einen vollständigen Satz zusammenhängender Datumsangaben. Das optionale Argument gibt den letzten Monat des Geschäftsjahres an:

```
Due Date = CALENDARAUTO(6)
```

Das Argument **6** legt Juni als letzten Monat des Geschäftsjahres fest. Die resultierende Tabelle enthält eine einzige Datumsspalte mit lückenlosen Werten vom Beginn bis Ende aller Jahre im Modell.

Hinweis Alternativ bietet **CALENDAR(Startdatum, Enddatum)** mehr Kontrolle — Start- und Enddatum können als statische Werte oder als Ausdrücke angegeben werden.

Screenshot Dateiname: dax-due-date-table-data-view-1.png — Datenansicht der Tabelle "Due Date" mit einer einzigen Datumsspalte, sortiert vom frühesten zum spätesten Datum (Beginn: 1. Juli 2017).

Als Datumstabelle markieren

Nach der Erstellung muss die Tabelle in Power BI Desktop als **Datumstabelle markiert** werden. Erst dann funktionieren DAX-Zeitintelligenzfunktionen korrekt. Power BI prüft bei der Markierung automatisch:

- Eindeutige Werte in der Datumsspalte

- Keine Nullwerte
- Lückenlos zusammenhängende Datumsangaben
- Gleicher Zeitstempel bei Datum/Uhrzeit-Werten

Diese Pflicht zur Markierung gilt für alle Datumstabellen — egal ob importiert, in Power Query erstellt oder als berechnete Tabelle.

Berechnete Spalten in der Praxis

Berechnete Spalten werden bevorzugt, wenn:

- Die Formel zusammengefasste Modelldaten benötigt
- Spezielle DAX-Modellierungsfunktionen wie **RELATED**, **RELATEDTABLE** oder Hierarchiefunktionen verwendet werden müssen
- Einer berechneten Tabelle weitere Spalten hinzugefügt werden sollen

In allen anderen Fällen ist eine **benutzerdefinierte Spalte in Power Query** die bessere Wahl — sie wird kompakter ins Modell geladen und belastet die Modellgröße weniger.

Beispiele: Datumstabelle erweitern

Die folgende Reihe von berechneten Spalten erweitert eine Datumstabelle um praxisrelevante Felder:

Geschäftsjahr (Geschäftsjahr beginnt im Juli):

```
Due Fiscal Year =  
"FY"  
    & YEAR('Due Date'[Due Date])  
    + IF(  
        MONTH('Due Date'[Due Date]) > 6,  
        1  
    )
```

Diese Formel verkettet "FY" mit dem Kalenderjahr und erhöht das Jahr für Monate Juli–Dezember um 1.

Geschäftsquartal:

```
Due Fiscal Quarter =  
'Due Date'[Due Fiscal Year] & " Q"  
    & IF(  
        MONTH('Due Date'[Due Date]) <= 3,  
        3,  
        IF(  
            MONTH('Due Date'[Due Date]) <= 6,  
            4,  
            IF(  
                MONTH('Due Date'[Due Date]) <= 9,  
                1,  
                2  
            )  
        )  
    )  
)
```

Monatsschlüssel (numerisch, für korrekte Sortierung von Monatsnamen):

```
MonthKey =  
(YEAR('Due Date'[Due Date]) * 100) + MONTH('Due Date'[Due Date])
```

Vollständige Datumsbeschriftung (lesbares Textformat):

```
Due Full Date =  
FORMAT('Due Date'[Due Date], "yyyy mmm, dd")
```

Screenshot Dateiname: dax-due-date-table-data-view-2.png — Datenansicht der Tabelle "Due Date" mit sechs Spalten: die originale Datumsspalte plus fünf berechnete Spalten.

Zeilenkontext verstehen

Berechnete Spaltenformeln werden für **jede Zeile** der Tabelle einzeln ausgewertet. Dieses Konzept heißt **Zeilenkontext** — die Formel arbeitet immer mit dem Wert der aktuellen Zeile.

Wenn eine Formel Werte aus einer **anderen Tabelle** benötigt, gibt es zwei Wege:

Situation	Funktion
Beziehung zwischen den Tabellen vorhanden	RELATED() (von der "Viele"-Seite zur "1"-Seite) oder RELATEDTABLE() (von der "1"-Seite zur "Viele"-Seite)

Keine Beziehung vorhanden

LOOKUPVALUE()

Hinweis **RELATED()** ist deutlich schneller als **LOOKUPVALUE()** — Power BI nutzt die vorhandenen Beziehungsindizes. Wenn möglich immer **RELATED()** bevorzugen.

```
Discount Amount =  
(  
    Sales[Order Quantity]  
    * RELATED('Product'[List Price])  
) - Sales[Sales Amount]
```

Der Zeilenkontext gilt auch in Iteratorfunktionen — dazu mehr im letzten Abschnitt dieses Skripts.

Implizite Measures

Implizite Measures sind automatische Aggregationsverhalten, die Power BI für numerische Spalten bereitstellt. Berichtsauctoren können eine Spalte in ein Visual ziehen und Power BI aggregiert sie automatisch — ohne dass ein explizites Measure geschrieben werden muss.

Das **Sigma-Symbol** (Σ) im Datenbereich zeigt an, dass eine Spalte numerisch ist und beim Hinzufügen zu einem Visual zusammengefasst wird.

Aggregationsfunktionen

Numerische Spalten unterstützen folgende Aggregationsfunktionen:

Sum, Average, Minimum, Maximum, Count (Distinct), Count, Standard deviation, Variance, Median

Nicht numerische Spalten (Text, Datum, Boolean) können ebenfalls aggregiert werden — allerdings mit eingeschränkten Optionen:

Spaltentyp	Mögliche Aggregationen
Text	First, Last, Count, Count (Distinct)

Datum	Earliest, Latest, Count, Count (Distinct)
Boolean	Count, Count (Distinct)

Einschränkungen impliziter Measures

Implizite Measures funktionieren nur für **einfache Aggregationen** — einen Spaltenwert mit einer einzigen Funktion zusammenfassen. Sobald eine Berechnung mehrere Spalten kombiniert, einen Quotienten berechnet oder zeitbasierte Logik benötigt, muss ein **explizites Measure** geschrieben werden.

Wichtig Implizite Measures lassen sich von Berichtsauctoren umprogrammieren — z. B. kann ein Berichtsauctor *Unit Price* auf Sum setzen, was zu irreführend hohen Werten führt (Summe aller Einzelpreise statt eines Durchschnittspreises). Empfehlung: Numerische Spalten mit expliziten Measures absichern und die Originalspalte ausblenden.

Explizite Measures erstellen

Explizite Measures werden über **Tabellentools** → **Neues Measure** angelegt und im Datenbereich mit dem Taschenrechnersymbol (田) angezeigt.

Einfache Measures

Ein einfaches Measure aggregiert die Werte einer einzelnen Spalte — verhält sich wie ein implizites Measure, ist aber im Modell fixiert:

```
Revenue =
SUM(Sales[Sales Amount])

Order Line Count =
COUNT(Sales[SalesOrderLineKey])

Order Count =
DISTINCTCOUNT('Sales Order'[Sales Order])
```

COUNTROWS() ist eine Alternative zu **COUNT()** und zählt direkt die Zeilen einer Tabelle:

```
Order Line Count =
COUNTROWS(Sales)
```

Hinweis Sofort nach dem Erstellen eines Measures die Formatierungsoptionen im Menüband **Measure-Tools** festlegen (Dezimalstellen, Währungssymbol, Prozent). Das stellt sicher, dass Werte in allen Visuals konsistent aussehen.

Verbundmeasures

Ein Verbundmeasure verweist auf ein oder mehrere andere Measures. Dadurch lassen sich komplexe Berechnungen aus einfacheren Bausteinen zusammensetzen:

```
Profit =  
[Revenue] - [Cost]  
  
Profit Margin =  
DIVIDE([Profit], [Revenue])
```

Wichtig Änderungen an einem Measure wirken sich auf alle abhängigen Measures aus. Verbundmeasures immer sorgfältig testen, bevor Änderungen eingecheckt werden.

Quickmeasures

Quickmeasures erlauben das Erstellen von Measures ohne DAX-Kenntnisse: Eine Berechnungsvorlage auswählen, Felder zuweisen — Power BI generiert die DAX-Formel automatisch. Die generierte Formel ist anschließend im Bereich **Daten** sichtbar und kann als Lerngrundlage dienen.

```
-- Von Quickmeasure generiertes Measure "Profit Margin":  
Profit Margin =  
DIVIDE([Profit], [Revenue])
```

Berechnete Spalten vs. Measures — der wichtigste Unterschied

	Berechnete Spalte	Measure
Zweck	Neues Attribut pro Zeile	Dynamische Aggregation über Modelldaten
Auswertung	Zeilenkontext bei Datenaktualisierung	Filterkontext zur Abfragezeit
Gespeichert	Ja — belastet Modellgröße	Nein — immer frisch berechnet

Visual-Nutzung	Filtern, Gruppieren, implizit aggregieren	Gezielt für Zusammenfassung
Leistung	Erhöht Speicher- und Aktualisierungslast	Effizienter bei großen Modellen
Ideal für	Statische Attribute, Beziehungsschlüssel	KPIs, Kennzahlen, dynamische Berechnungen

Iteratorfunktionen

Iteratorfunktionen werten einen Ausdruck für **jede Zeile einer Tabelle** aus und aggregieren die Ergebnisse. Sie sind erkennbar am „X“-Suffix: **SUMX**, **AVERAGEX**, **COUNTX**, **MINX**, **MAXX**.

Jede Iteratorfunktion benötigt zwei Argumente:

1. Eine **Tabelle** (Modelltabelle oder tabellenrückgebender Ausdruck)
2. Einen **Ausdruck**, der für jede Zeile einen Einzelwert zurückgibt

Einfacher Vergleich: SUM vs. SUMX

SUM ist eine Abkürzung für **SUMX** — Power BI konvertiert es intern:

```
-- Identische Ergebnisse und identische Leistung:
Revenue = SUM(Sales[Sales Amount])

Revenue =
SUMX(
    Sales,
    Sales[Sales Amount]
)
```

Komplexe Zusammenfassung über mehrere Spalten

Der eigentliche Mehrwert von Iteratorfunktionen liegt in der Möglichkeit, pro Zeile einen Ausdruck aus mehreren Spalten zu berechnen und dann zu aggregieren:

```
Revenue =
SUMX(
```

```

Sales,
Sales[Order Quantity] * Sales[Unit Price] * (1 - Sales[Unit Price Discount Pct])
)

```

Mit **RELATED** kann in der Iterationsformel auch auf Spalten aus einer verwandten Tabelle zugegriffen werden:

```

Discount =
SUMX(
    Sales,
    Sales[Order Quantity]
    * (
        RELATED('Product'[List Price]) - Sales[Unit Price]
    )
)

```

Screenshot Dateiname: dax-table-month-revenue-2.png — Tabellenvisual mit den Spalten Month, Revenue und Discount auf Basis der SUMX-Measures.

Granularität steuern mit AVERAGEX

Iteratorfunktionen können auch auf verschiedenen Granularitätsebenen aggregieren. Mit **VALUES()** lässt sich die Iteration auf eindeutige Werte einer Spalte beschränken:

```

-- Durchschnittsumsatz pro Auftragsposition:
Revenue Avg Order Line =
AVERAGEX(
    Sales,
    Sales[Order Quantity] * Sales[Unit Price] * (1 - Sales[Unit Price Discount Pct])
)

-- Durchschnittsumsatz pro Verkaufsauftrag (höheres Aggregationsniveau):
Revenue Avg Order =
AVERAGEX(
    VALUES('Sales Order'[Sales Order]),
    [Revenue]
)

```

VALUES() gibt die eindeutigen Verkaufsauftragsnummern im aktuellen Filterkontext zurück. **AVERAGEX** durchläuft dann jeden Auftrag und berechnet den Durchschnitt des Gesamtumsatzes.

Screenshot Dateiname: dax-table-month-revenue-4.png — Tabellenvisual mit fünf Spalten: Month, Revenue, Discount, Revenue Avg Order Line, Revenue Avg Order.

Rangfolge mit RANKX

RANKX berechnet einen Rang, indem es eine Tabelle durchläuft und für jede Zeile einen Ausdruck auswertet. Standardmäßig absteigende Reihenfolge (höchster Wert = Rang 1):

```
Product Quantity Rank =  
RANKX(  
    ALL('Product'[Product]),  
    [Quantity]  
)
```

ALL() entfernt alle Filter, sodass alle Produkte gemeinsam gerankt werden — unabhängig vom aktuellen Filterkontext des Visuals.

Dense-Rangfolge (keine Ränge überspringen bei Gleichstand):

```
Product Quantity Rank =  
RANKX(  
    ALL('Product'[Product]),  
    [Quantity],  
    ,  
    ,  
    DENSE  
)
```

Gesamtzeile unterdrücken mit **HASONEVALUE**:

```
Product Quantity Rank =  
IF(  
    HASONEVALUE('Product'[Product]),  
    RANKX(  
        ALL('Product'[Product]),  
        [Quantity],  
        ,  
        ,  
        DENSE  
    )  
)
```

HASONEVALUE() prüft, ob die Produktspalte im aktuellen Filterkontext genau einen Wert enthält. Für die Gesamtzeile, die alle Produkte repräsentiert, ist das nicht der Fall — das Measure gibt dann BLANK zurück.

Wichtig Iteratorfunktionen mit großen Tabellen und komplexen Ausdrücken können die Abfrageleistung erheblich beeinflussen. Funktionen wie **SEARCH()** und **LOOKUPVALUE()** im

Iterationsausdruck sind kostspielig — wenn möglich **RELATED()** bevorzugen.

Zusammenfassung

Datumstabellen

Mit CALENDARAUTO() oder CALENDAR() als berechnete Tabelle erzeugen. Anschließend als Datumstabelle markieren — zwingend für Zeitintelligenz. Muss eindeutige, lückenlose Datumsangaben enthalten.

Berechnete Spalten

Für Attribute, die zusammengefasste Daten oder DAX-Modellierungsfunktionen (RELATED, RELATEDTABLE) benötigen. Zeilenkontext gilt immer. In Power Query-Spalten bevorzugen wenn möglich — geringere Modellgröße.

Σ

Implizite Measures

Automatische Aggregation numerischer Spalten. Flexibel, aber riskant — Berichtsauctoren können die Aggregationsmethode ändern. Numerische Schlüsselspalten mit expliziten Measures absichern.

田

Explizite Measures

Einfach (eine Spalte aggregieren), Verbund (andere Measures referenzieren) oder über Quickmeasures erstellen. Format sofort nach Erstellung festlegen. Measures niemals mit Tabellenpräfix referenzieren.

Iteratorfunktionen

SUMX, AVERAGEX, COUNTX, MINX, MAXX — werten einen Ausdruck zeilenweise aus. Ermöglichen mehrspaltige Berechnungen und Granularitätskontrolle. RANKX für Rangfolgen. Spalte oder Measure?

Berechnete Spalte: statisches Attribut, das für Slicing oder Beziehungen benötigt wird.

Measure: dynamische Berechnung, die vom Filterkontext abhängt. Im Zweifelsfall Measure bevorzugen — geringere Modellgröße.



Modul 3 – Semantisches Modell & DAX
DAX – Datentypen, Funktionen,
Operatoren und Variablen

Modul 3 – Semantisches Modell & DAX
Filterkontext & CALCULATE



